

Bellerophon Language Documentation

Language Overview

Bellerophon is a domain-specific language (DSL) for 3D printing. Write clean, procedural motions and parametric macros that compile directly to any firmware dialect for additive manufacturing.

Note:

Philosophy: Precise, readable commands that make controlling 3D printers easier.

Getting Started

Bellerophon is a compiled Domain-Specific Language (DSL) and IDE for additive manufacturing. It allows you to write structured, parametric scripts and compile them into firmware-compatible G-code macros. This section describes the setup and workflow for the Bellerophon development environment.

1. System Requirements

- **Bundle:** Bellerophon includes a bundled JRE, eliminating the need for manual Java installation.
- **Hardware:** Any modern Windows, Linux, or macOS system.

2. Launching the IDE

The Bellerophon IDE is delivered as a self-contained environment. Simply launch the provided executable to initialize the local server.

Windows

- Navigate to the project root folder.
- Double-click `run.bat`. This will initialize the bundled JRE and start the Bellerophon server.
- The IDE will automatically launch in your default web browser at `http://localhost:4567`.

- To shut down, simply close the server terminal window.

Linux / Mac

- Navigate to the project root folder in your terminal.
- Ensure the launch script is executable: `chmod +x run.sh`
- Run the script: `./run.sh`
- The IDE will be accessible at `http://localhost:4567`.
- Press `Ctrl+C` in the terminal to terminate the server.

Note:

The server runs locally. By default, it listens on port **4567**, but this can be changed by editing the `config.properties` file. No internet connection is required after installation.

3. Compilation Architecture

Bellerophon uses a decoupled **Firmware Adapter Architecture**. Your `.bph` source code remains universal, while the compiler handles the conversion to firmware-specific instructions via targeted adapters.

- **Universal Source:** Write once, deploy anywhere. Your logic is maintained in a single Bellerophon project regardless of the printer controller.
- **Modular Adapters:** Choose your target in the IDE prior to compilation. The compiler applies the appropriate syntax rules and feature-set limitations for that specific environment (e.g., Klipper, Marlin, or future custom firmware targets).
- **Firmware Independence:** The Bellerophon engine is designed to be firmware-agnostic; as new manufacturing controllers emerge, new adapters can be integrated without modifying your existing scripts.

4. IDE & Workflow

- **Input:** Bellerophon processes `.bph` scripts. The IDE provides real-time, line-oriented syntax validation and error highlighting as you develop.
- **Debugging:** Access the **Build Log** and **Exceptions** tabs for immediate feedback on compilation errors.
- **Session Management:** The IDE automatically preserves your workspace, restoring open files and settings to prevent data loss.
- **Reference Panel:** Access all Bellerophon tokens and documentation directly within the IDE sidebar to reduce syntax lookups and speed up your workflow.

5. Core Capabilities

Bellerophon is a **Parametric Engine**, moving manufacturing beyond static G-code into procedurally generated motion.

Parametric Geometry

Use global variables, nested Brepeat loops, and advanced math (sin, cos, sqrt, pi) to define complex motion paths programmatically.

Smart Extrusion

Utilize the EnableAutoExtrude engine for path-aware volume calculation, or perform manual overrides for high-precision extrusion control.

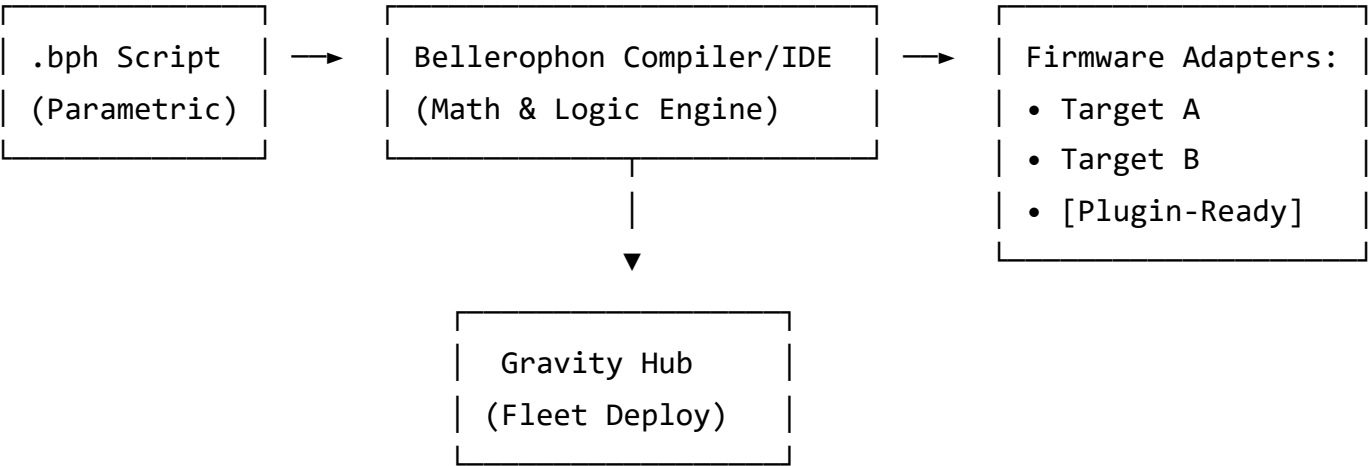
Because these features are evaluated by the **Bellerophon Compiler** at build-time, your final macro output is optimized, validated, and ready for hardware execution. This offloads the computational weight of complex mathematics from your printer's controller, ensuring fluid motion even for highly intricate, procedural geometry.

6. Technical Considerations & Limitations

- **DSL Purpose:** Bellerophon is a compilation engine, not a 3D slicer; it does not process 3D meshes (STL/STEP).
- **Variable Scope:** Bellerophon supports both local variables (scoped to the macro) and global variables (defined with var)
- **Target Compatibility:** Feature support (such as conditional logic) is determined by the specific capabilities of the firmware target adapter selected at compile-time.

7. Bellerophon Architecture & Workflow

Bellerophon operates as a high-level parametric engine, abstracting complex geometry into firmware-specific instructions via a modular adapter system.



Note:

Extensible Architecture: Bellerophon uses a decoupled compiler design. New firmware targets can be added by implementing a new `FirmwareAdapter`, allowing the DSL to remain constant even as the manufacturing landscape changes.

Program Structure

A Bellerophon program consists of one or more macros. Each macro starts with `M.title` and ends with `M.end` .

```
M.title "Macro Name"
```

```
    [statement 1]
```

```
    [statement 2]
```

```
    [statement 3]
```

```
M.end
```

Each command must be on its own line. Indentation is optional but recommended. Comments start with `#` .

User Variables (Local & Global)

Bellerophon supports both **local variables** (defined inside a single macro) and **global variables** (defined with the `var` keyword and shared across macros).

Note:

Scope Rules: Variables defined without `var` are local to the macro and cleared when the macro ends. Variables defined with `var` are global and persist across all macros. If a local and global variable share the same name, the **local variable shadows (takes priority over) the global** inside that macro. Undefined variables default to `0.0`.

Local Variables (Standard Assignment)

To define or update a variable that only exists inside the current macro:

```
M.title "Main"
```

```
    pos_x = 50
```

```
    pos_y = pos_x + 10
```

```
    MoveTo x=pos_x y=pos_y
```

```
M.end
```

● *These variables are discarded when the macro ends.*

Global Variables (var)

To define a variable that persists across the entire file and can be shared between macros, use the `var` keyword. A global variable defined in one macro can be read or modified by any other macro in the same compilation unit:

```
M.title "MacroA"
    var pos_x = 50
    var pos_y = 60
    SetSpeed = 3000
    MoveTo x=pos_x y=pos_y
M.end
```

```
M.title "MacroB"
    pos_x = pos_x + 10    # Now 60
    pos_y = pos_y + 10    # Now 70
    MoveTo x=pos_x y=pos_y
M.end
```

● *Global variables persist across the entire file. `pos_x` starts at 50 and `pos_y` at 60 in MacroA, then both become 60 and 70 in MacroB.*

Assignment Syntax

To define or update a variable, use the assignment statement:

```
variable_name = expression
```

Supported Expressions

Variables can be assigned any valid expression:

- **Numbers:** 150, 3.14, -5
- **Arithmetic:** +, -, *, /, ^ (power)
- **Parentheses:** (base + offset) * 2
- **Functions:** `sin()`, `cos()`, `tan()`, `sinr()`, `cosr()`, `tanr()`, `sqrt()`, `abs()`, `sign()`, `deg()` — `sin/cos/tan` take degrees, `sinr/cosr/tanr` take radians, `deg()` converts radians to degrees
- **Radian functions:** `sinr()`, `cosr()`, `tanr()` take radians instead of degrees. `deg()` converts a radian value back to degrees.

- **Constant:** pi
- **Variable references:** my_x, speed (must be defined earlier in the macro)

Example

```
M.title "example"
  base = 100
  offset = 50
  speed = (base + offset) * 2
  SetSpeed = speed
  MoveTo x=base y=offset
  MoveTo x=base+offset y=base-offset
M.end
```

Output

```
[gcode_macro example]
gcode:
  G1 F300
  G1 X100.000 Y50.000
  G1 X150.000 Y50.000
```

Statement Reference

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
<i>Assignment</i>	<i>variable</i> = expression	<i>my_x</i> = 150	Stores the expression result in a local variable. Variables are scoped to the macro. Supports arithmetic operations: +, -, *, /, ^ (power), and functions: sin(), cos(), tan(), sinr(), cosr(), tanr(), sqrt(), abs(), sign(), deg() (radians to degrees), and constant pi.
Home	Home OR Home -> coordList	Home OR Home -> x=0 y=0	Homes the printer. If coordinates are provided, only the specified axes are homed. Without

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			coordinates, all axes are homed.
Move	Move direction	Move left / right / center / up / down	Moves the toolhead 1mm in the specified direction. center moves to X0 Y0. Directional moves are relative to the current position.
MoveTo	MoveTo coordList	MoveTo x=100 y=100 z=0.2	Moves the toolhead to the specified coordinates. Supports X, Y, Z, and E (extrusion) axes. If EnableAutoExtrude is active and no manual E value is provided, extrusion is calculated automatically based on distance traveled and printer settings.
Absolute	Absolute	Absolute	Sets positioning to absolute mode. All coordinates are relative to the origin (0,0,0). This is the default mode.
Relative	Relative	Relative	Sets positioning to relative mode. All coordinates are offsets from the current position. Essential for creating flowing, iterative shapes in loops.
Heat	Heat target=number	Heat extruder=200	Sets the temperature of the specified heater and waits until the target temperature is reached. Targets: extruder, bed, chamber. This blocks further execution until the temperature is stable.
Set_Heater_Temperature	Set_Heater_Temperature target=number	Set_Heater_Temperature bed=60	Sets the temperature of the specified heater but does NOT wait for it to reach the target. Execution

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			continues immediately. Targets: extruder, bed, chamber.
WaitForTemp	WaitForTemp target	WaitForTemp extruder	Waits until the specified heater reaches its currently set temperature. Does not change the temperature. Useful after a Set_Heater_Temperature command to pause execution until heating completes.
Cooldown	Cooldown [target]	Cooldown extruder Cooldown (no target = all)	Turns off heaters. With a target, turns off only that specific heater (extruder, bed, chamber). Without a target, turns off all heaters. This is a safety command to prevent overheating after a print or macro completes.
TimeoutSet	TIMEOUT_SET seconds	TIMEOUT_SET 300	Sets the idle timeout in seconds. After this period of inactivity, the printer will automatically disable motors and heaters to save power and prevent wear. Set to 0 to disable the timeout.
Pause	Pause	Pause	Pauses the current print or macro execution. Useful for filament changes, manual interventions, or waiting for user input. Can be resumed with Resume.
Resume	Resume	Resume	Resumes a paused print or macro execution. Continues from where the pause occurred.
Respond	Respond MSG "text"	Respond MSG "Print complete"	Sends a message to the printer's display or console log. Useful for status

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			updates, notifications, or debugging.
<code>SetSpeed</code>	<code>SetSpeed = number</code>	<code>SetSpeed = 100</code>	Sets the movement speed in mm/min for all subsequent moves. This affects travel moves, extrusion moves, and any coordinated motion.
<code>SetFan</code>	<code>SetFan = number</code>	<code>SetFan = 255</code>	Sets the cooling fan speed. Value range depends on firmware (typically 0-255 or 0-100%). 0 turns the fan off, maximum value runs the fan at full speed.
<code>SetNozzle</code>	<code>SetNozzle = number</code>	<code>SetNozzle = 0.6</code>	Sets the nozzle diameter (mm) for extrusion calculations. This affects the volume of filament extruded during auto-extrusion. Default is taken from the printer profile UI. Compile-time only: This is a compiler setting, not a G-code command.
<code>EnableAutoRetract</code>	<code>EnableAutoRetract = 1 0</code>	<code>EnableAutoRetract = 1</code>	Enables (1) or disables (0) automatic retraction on qualifying XY travel moves. Uses retraction distance, speed, and minimum travel from the printer profile. Requires <code>RelativeExtrusion</code> in the macro before a move that triggers retraction. Compile-time only.
<code>SetFilament</code>	<code>SetFilament = number</code>	<code>SetFilament = 2.85</code>	Sets the filament diameter (mm) for extrusion calculations. This affects the volume of filament extruded during auto-extrusion. Default is taken from the printer profile UI.

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			Compile-time only: This is a compiler setting, not a G-code command.
<code>SetLayerHeight</code>	<code>SetLayerHeight = number</code>	<code>SetLayerHeight = 0.3</code>	Sets the layer height (mm) for extrusion calculations and the Layer command. This affects the Z increment per layer and the volume of filament extruded. Default is taken from the printer profile UI. Compile-time only: This is a compiler setting, not a G-code command.
<code>SetExtrusionMultiplier</code>	<code>SetExtrusionMultiplier = number</code>	<code>SetExtrusionMultiplier = 0.95</code>	Adjusts the extrusion multiplier (flow rate compensation). Values below 1.0 reduce extrusion, values above 1.0 increase extrusion. Default is taken from the printer profile UI. Compile-time only: This is a compiler setting, not a G-code command.
<code>EnableAutoExtrude</code>	<code>EnableAutoExtrude = 1 0</code>	<code>EnableAutoExtrude = 1</code>	Enables (1) or disables (0) automatic extrusion calculation for MoveTo commands. When enabled, extrusion is calculated based on distance traveled and printer settings (nozzle diameter, filament diameter, layer height, extrusion multiplier). Manual e= . . . in a MoveTo command overrides auto-extrusion. Compile-time only: This is a compiler setting, not a G-code command.

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
Layer	Layer number ... end	Layer 5 ... end	Repeats the enclosed block for the specified number of layers, automatically raising Z by the current LayerHeight after each layer. This is useful for creating multi-layer structures procedurally. Note: Z-axis movement is not allowed inside Layer blocks - the Layer command manages Z automatically.
PrintFile	PRINTFILE "file.gcode"	PRINTFILE "cube.gcode"	Selects and starts printing the specified G-code file from the printer's SD card or storage. The file must exist on the printer's storage medium.
Call Macro	M.call "MacroName"	M.call "CalibrateBed"	Calls another macro by name. The called macro must be defined elsewhere in the program. Useful for organizing code into reusable modules.
If / EndIf	if condition ... endif	if extruder > 190 ... endif	Conditional execution block. The condition evaluates using printer sensor values at runtime. Valid sensors: extruder, bed, chamber. Supports comparison operators: >, <, >=, <=, ==, !=.
RelativeExtrusion	RelativeExtrusion	RelativeExtrusion	Sets extrusion to relative mode. All E values are treated as increments from the current extruder position. This is useful for consistent extrusion in repeated patterns.
Retract	Retract [distance]	Retract Retract 5	Retracts the printer-profile distance, or the optional

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			positive distance in millimeters. Retraction speed always comes from the printer profile. Requires RelativeExtrusion in the macro.
<code>Unretract</code>	<code>Unretract [distance]</code>	<code>Unretract</code> <code>Unretract 5</code>	Unretracts the profile distance, or the optional positive distance in millimeters. Retraction speed always comes from the printer profile. Requires RelativeExtrusion in the macro.
<code>ResetExtruder</code>	<code>ResetExtruder</code>	<code>ResetExtruder</code>	Resets the extruder position to zero (G92 E0). This is typically done at the start of a print or after a filament change.
<code>BedMeshCalibrate</code>	<code>BedMeshCalibrate</code>	<code>BedMeshCalibrate</code>	Performs bed mesh calibration to map the bed surface and compensate for unevenness. The printer probes multiple points and creates a height map for Z compensation during printing.
<code>Level</code>	<code>Level</code>	<code>Level</code>	Alias for BedMeshCalibrate. Performs bed mesh calibration.
<code>ProbeCalibrate</code>	<code>ProbeCalibrate</code>	<code>ProbeCalibrate</code>	Calibrates the Z-probe offset for accurate first layer height. This adjusts the Z coordinate at which the probe triggers, ensuring the nozzle is at the correct height above the bed.
<code>LoadBedMesh</code>	<code>LoadBedMesh "profile"</code>	<code>LoadBedMesh "default"</code>	Loads a previously saved bed mesh profile by name. This applies the stored

COMMAND	SYNTAX	EXAMPLE	DESCRIPTION
			height map without re-probing the bed.
<code>SetPressureAdvance</code>	<code>SetPressureAdvance</code> number	<code>SetPressureAdvance</code> 0.04	Sets the pressure advance / linear advance value for improved extrusion consistency. This compensates for the delay between extruder movement and actual filament flow.
<code>Dwell</code>	<code>Dwell</code> time S ms (optional unit)	<code>Dwell</code> 0.5 S or <code>Dwell</code> 500 ms	Pauses execution for the specified duration. If unit is omitted, defaults to seconds. S indicates seconds, ms indicates milliseconds.
<code>InsertGcode</code>	<code>InsertGcode</code> "filename.gcode" <code>InsertGcode</code> "filename.gcode" reference	<code>InsertGcode</code> "part_A.gcode" <code>InsertGcode</code> "part_B.gcode" reference	Includes an external G-code file. In embed mode (default), the compiler reads the file and injects its contents into the output. In reference mode, the compiler emits a firmware command to reference the file on the printer's SD card.

Advanced Features & Concepts

1. Parametric Geometry Engine

Bellerophon goes beyond simple macro scripts by acting as a **Parametric DSL**. Because the compiler evaluates mathematical expressions at build-time, you can create complex, procedural geometry that scales based on variables or mathematical functions.

- 🌟 **Global Math:** Use `sin`, `cos`, `tan`, `sqrt`, `abs`, `sign`, `^` (power), and `pi` anywhere an expression is expected (e.g., coordinates, temperatures, or speed).

- **Procedural Motion:** By combining Brepeat loops with trigonometric functions, you can generate spirals, waves, and curves that are impossible to maintain by hand.
- **Expression Evaluation:** All mathematical expressions are evaluated during compilation, not at runtime. This means complex calculations don't burden the printer's controller.

2. The "Override" System (Auto vs. Manual)

Bellerophon employs an intelligent extrusion engine designed to simplify your workflow while retaining full manual control:

Note:

Execution Priority:

1. **Manual Override:** If an `e=...` value is provided in a MoveTo command, it is **always** prioritized. The value is multiplied by the current extrusion multiplier.
2. **Automatic Calculation:** If `EnableAutoExtrude = 1` is active and no manual `e` is provided, the compiler calculates the required filament volume based on your defined nozzle diameter, filament diameter, and layer height. The formula used is: $E = \text{distance} * (\text{nozzle_diameter}^2 / \text{filament_diameter}^2) * \text{extrusion_multiplier}$.
3. **Travel Move:** If `EnableAutoExtrude = 0` and no `e` is provided, the printer performs a pure travel move (no extrusion).

The extrusion calculation uses the current printer settings (nozzle diameter, filament diameter, layer height, extrusion multiplier). These can be overridden in the macro using `SetNozzle`, `SetFilament`, `SetLayerHeight`, and `SetExtrusionMultiplier`.

3. Retraction

`EnableAutoRetract = 1` enables compile-time retraction around eligible travel moves. Retraction values come from the active printer profile: retraction distance (mm), retraction speed (mm/s), and minimum travel distance (mm).

- The threshold uses XY Euclidean distance: $\sqrt{dx^2 + dy^2}$. Z movement does not contribute.
- A retract is considered only after a positive extrusion has occurred in the macro. This prevents retracting on the initial positioning move.
- Moves with an explicit `e` value or automatic extrusion are not treated as travel moves.
- Each qualifying travel is wrapped with a profile-distance retract before the move and a matching unretract after it.

Automatic and manual retraction require `RelativeExtrusion` in the macro. For automatic retraction, it must appear before the first travel move that triggers a retract. If it is missing entirely, compilation reports an error at the `EnableAutoRetract` setting. Manual `Retract` and `Unretract` use the profile distance by default; an optional positive distance overrides that value while retaining profile speed, for example `Retract 5`. Zero and negative overrides are rejected.

4. Procedural Layering (`Layer`)

The `Layer` command automates vertical iteration. The `Layer` command executes a block once per printed layer and automatically advances the Z-axis using the configured `LayerHeight`.

Layer 50

```
MoveTo x=100 y=50
```

```
MoveTo x=120 y=50
```

```
end
```

Note:

Important: Z-axis movement is not allowed inside `Layer` blocks. The `Layer` command manages Z automatically by incrementing the Z position by `LayerHeight` at the start of each layer.

This is beneficial for procedural planar slicing, allowing you to build up volume automatically without manually managing Z-heights.

5. Compile-Time vs Run-Time

Understanding when things happen is crucial:

- **Compile-Time (Bellerophon Compiler):** Math expressions, variable assignment, loop unrolling, auto-extrusion calculations, and printer setting modifications all happen during compilation. The output G-code contains pre-calculated values.
- **Run-Time (Printer Firmware):** Only G-code commands are executed. The printer does not know about Bellerophon variables or expressions.

Note:

Why this matters: This design offloads computational weight from your printer's controller, ensuring fluid motion even for complex procedural geometry. The printer only receives pre-calculated movement commands.

6. Thermal Management (Cooldown)

The `Cooldown` command provides flexible temperature control:

```
# Turn off all heaters
```

```
Cooldown
```

```
# or explicitly:
```

```
Cooldown all
```

Turn off specific heaters

Cooldown extruder

Cooldown bed

Cooldown chamber

Behavior:

- **No target** → Turns off all heaters (extruder, bed, chamber)
- **Specific target** → Only turns off that specific heater

7. Variable Scoping

Bellerophon supports two types of variables with different scope rules:

- **Local variables** (defined without `var`) are scoped to the macro they are defined in. They are created when the macro starts and discarded when the macro ends. This prevents name collisions across different macros.
- **Global variables** (defined with the `var` keyword) persist across the entire compilation unit. They can be accessed and modified from any macro in the same file.

Shadowing Priority: If a local variable and a global variable share the same name, the **local variable takes priority** inside that macro. The global variable is temporarily hidden and becomes visible again once the macro ends.

Default Value: If a variable name is used but has not been defined either locally or globally, the compiler defaults to `0.0`.

Note:

- Local (no `var`) → macro-only, cleared at end.
- Global (`var`) → whole file, persists everywhere.
- Local always beats global when names conflict.
- Undefined variables → `0.0`.

8. Assignment Operators

Operators allow for precise control during movement and variable assignment:

- `=` Direct Assignment - sets a variable to a specific value
- `+=` Add to current - adds a value to the current variable
- `-=` Subtract from current - subtracts a value from the current variable
- `*=` Multiply current - multiplies the current variable by a value
- `/=` Divide current - divides the current variable by a value

These operators work on both variables and coordinates in MoveTo commands.

Loops & Arithmetic Operators

Bellerophon supports two types of loops: `repeat` for static repetition and `Brepeat` for dynamic loops.

Note:

Positioning Context in Loops: The behavior of movement inside a loop depends heavily on the active mode:

- ➡ **Relative (G91):** Coordinates are calculated as offsets from the last position. Each move builds on the previous one. Best for iteratively forming shapes, spirals, zig-zags, or any flowing patterns.
- ➡ **Absolute (G90):** Coordinates are calculated from a fixed origin on the bed. Each move goes to a specific point. Use this only when you need precise positions that don't depend on the previous move, like moving to a center point or fixed markers.

Example: Absolute vs Relative in Loops

The same loop code produces very different results depending on the active mode:

Absolute Mode Example

Absolute

MoveTo x=10

Brepeat 4

 MoveTo x+=5

end

Relative Mode Example

Relative

MoveTo x=10

Brepeat 4

 MoveTo x+=5

end

Output:

Absolute Mode Output

```
[gcode_macro Absolute!-tester]
```

```
gcode:
    G90
    G1 X10.000
    G1 X15.000
    G1 X20.000
    G1 X25.000
    G1 X30.000
```

Relative Mode Output

```
[gcode_macro Relative!-tester]
```

```
gcode:
    G91
    G1 X10.000
    G1 X5.000
    G1 X5.000
    G1 X5.000
    G1 X5.000
```

Note:

Key observation: In absolute mode, `x+=5` calculates a new absolute position (10+5=15, then 15+5=20, etc.). In relative mode, it adds 5 to the current position each time, creating equal-sized steps from wherever the previous move ended.

1. Static Repeat (`repeat`)

The `repeat` command simply duplicates the enclosed G-code commands a specified number of times during compilation. The value must be a constant number, not a variable or expression.

```
repeat 3
    MoveTo x+=10 y+=10
    Respond MSG "Moving..."
end
```

Output (repeat):

```
G1 X10.0 Y10.0
RESPOND MSG="Moving..."
G1 X20.0 Y20.0
RESPOND MSG="Moving..."
```

```
G1 X30.0 Y30.0
```

```
RESPOND MSG="Moving..."
```

Note:

The commands are physically duplicated in the output G-code. Great for simple, fixed repetitions where the number of iterations is known at compile-time.

2. Dynamic Loop (`Brepeat`)

The `Brepeat` command creates a loop that supports arithmetic expressions and mathematical functions. Unlike `repeat`, this loop exposes an iterator variable `i` which increments each iteration starting from 0. Expressions are evaluated during compilation, producing numeric G-code values.

Note:

Important: The iterator `i` is only valid inside `Brepeat` loops, but **mathematical functions (`sin`, `cos`, `tan`, `sqrt`, `abs`, `sign`) and the power operator (^) are globally available anywhere an expression is allowed** (e.g., in `MoveTo`, `Heat`, `SetSpeed`, etc.).

Example: Using the iterator

```
Brepeat 5
    MoveTo x=i
end
```

Example: Mathematical Expressions (global availability)

Bellerophon supports arithmetic expressions anywhere: including exponentiation `^`, absolute value `abs()`, signum `sign()`, and the constant `pi`.

```
Brepeat 3
    MoveTo x=i^2 * 5 + abs(i-2)
end
Heat extruder=180 + 20*sin(90/2)
# works outside loops too
```

Example: Drawing a Square (Relative)

Using **Relative** mode ensures each move is from the previous point, naturally forming shapes when iterating.

Relative

Brepeat 4

MoveTo x=10 y=0

MoveTo x=0 y=10

MoveTo x=-10 y=0

MoveTo x=0 y=-10

end

Example: Drawing a Circle (Relative)

Using **Relative** mode allows the circle to be built incrementally, keeping each move relative to the last point for smooth motion.

Relative

Brepeat 60

MoveTo x=cos(i*0.1)*5 y=sin(i*0.1)*5

end

Example: Drawing a Spiral (Relative)

Using **Relative** mode allows complex parametric shapes to be built incrementally. This spiral uses the loop counter *i* to calculate both angle and radius, creating an expanding spiral pattern.

Spiral test macro

M.title "spiral_test"

MoveTo x = 160 y = 160 z= 10

Relative

SetSpeed = 6500

Brepeat 65

MoveTo x=cos(i*10)*(i*0.1) y=sin(i*10)*(i*0.1)

end

Dwell 2 ms

Absolute

MoveTo x = 150 y = 150

Dwell 2 s

Home

M.end

Note:

How the spiral formula works: The expression `cos(i*10)*(i*0.1)` and `sin(i*10)*(i*0.1)` create an expanding spiral where:

- `i*10` controls the angular step (angle increases quickly, making many turns)
- `i*0.1` controls the radius growth (spiral expands outward slowly)
- The product creates a spiral that grows in radius as it rotates
- Using **Relative** mode means each move is an offset from the previous position
- This creates a smooth, continuous spiral path

Nested Brepeat Loops

Bellerophon supports nested Brepeat loops. Each nested loop has its own iterator `i` that is scoped to that loop. This allows for complex 2D and 3D patterns:

```
Brepeat 5
  Brepeat 5
    MoveTo x=i*10 y=i*10
  end
end
```

Example Program

```
M.title "Bed Leveling Routine"
Absolute
SetSpeed = 3000
Home -> x=0 y=0
MoveTo z=5
Heat bed=60
Respond MSG "Bed heating complete"
Level
LoadBedMesh "default"
MoveTo x=100 y=100
ProbeCalibrate
SetPressureAdvance 0.04
ResetExtruder
RelativeExtrusion
Move right
Move up
Pause
```

Resume

Cooldown

M.end

External G-code Insertion (InsertGcode)

The `InsertGcode` command allows you to include external G-code files in your BPH script. This is useful for print farm loops, wrapping existing slicer outputs, and modular print workflows.

File Location

`InsertGcode` does not look for files relative to your `.bph` script. Instead, it reads from a single **G-code Library folder** that you set once in the IDE's Settings panel.

Open **Settings** → **G-Code Library** and type the full path to a folder on your machine (e.g. `D:/gcode_folder`). The IDE scans that folder for `.gcode`, `.g`, and `.gc` files and builds a list of what it finds. That list is what powers autocomplete when you type a filename inside an `InsertGcode` string, and it's also what `InsertGcode` checks against at compile time.

Dual-Mode Operation

The command operates in two modes:

- **Embed Mode (default):** `InsertGcode "file.gcode"`
 - Reads the file at **compile time** and injects its raw G-code directly into the compiler's output stream.
 - Best for small to medium-sized files.
- **Reference Mode:** `InsertGcode "file.gcode" reference` OR `InsertGcode reference "file.gcode"`
 - Emits optimized firmware instructions (e.g., M98 for Klipper, M23 for Marlin) to target the printer's local SD card directory.
 - Use this mode for **large files** to avoid performance issues.

Example Usage

```
# Include a small G-code file directly
InsertGcode "part_A.gcode"
```

Include a large G-code file by reference

InsertGcode "part_B.gcode" *reference*